Usage & Packaging (macOS .dmg / Windows .exe)

Goal: students should not need to install Python. Double-click to launch.

How to Use (Students / Teachers)

- Typical workflow:
  - Home → `New Project` → `Image`
  - Choose a start mode:
    - `Start from empty`: create classes and collect samples from scratch
    - `Start from classified class`: import a dataset that is already grouped by label
    - `Open .tmproj`: restore a previously saved project archive
- Workspace navigation:
  - Open the top-left `Teachable Machine` menu for `Open project`, `Save project`, `Return`, and `Reset project`
  - `Return` goes back to the page you came from:
    - empty project workspace → home
    - classified import workspace → classified import page
    - opened `.tmproj` workspace → home
  - `Reset project` clears the current workspace session
- Classes and samples:
  - `+ Add a class`: add a new class card
  - Click the class name to rename it
  - Each class card supports `Webcam`, `Upload`, and `Device`
  - `Webcam` and `Upload` samples are shown in color in the UI when available
  - `Device` samples are grayscale frames from a serial/UART source
  - Training input is normalized to grayscale for all sample sources, so webcam/upload/device are trained consistently
- Device source: - Open the `Device` card and use the ⚙ settings panel to configure `Image Size`, `Color Mode`, `Baud Rate`, and `Sync Header` - `Image Size` selects the frame resolution sent by the firmware: 96×96, 160×160, or 384×384 — must match the firmware output exactly - `Color Mode` selects Grayscale (1 channel) or RGB (3 channels) depending on the firmware sketch - `Sync Header` is the hex frame marker used to detect the start of each image packet, for example `AA 55 AA` - Change the sync value if your firmware uses a different frame prefix
- Classified import:
  - `Browse...` opens the system folder picker
  - `Load folder` stays disabled until a folder path is present
  - Supported input layouts are documented below in `Dataset Folder Structure`
  - Imported class names come from folder / label names
- Training:
  - The middle `Training` panel trains an int8 TFLite model from the current workspace
  - The `Advanced` section exposes all model hyperparameters: `Image Size` (model input resolution), batch size, epochs, learning rate, conv/dense layer sizes
  - `Image Size` controls the resolution images are resized to before training — all samples are preprocessed to this size regardless of their original dimensions

    - Image preprocessing is configured per class from the preprocess button shown next to each class name
    - The class preprocess `Processed Preview` shows the thresholded mask — white = filtered out (background), dark = candidate sign pixels. Tune the `Dark Thresh` and `Lum Thresh` sliders in the edit page to adjust what the detector keeps
    - Each class preprocess page supports:
        - `Manual ROI`: draw a custom ROI directly on a sample image for that class
        - `Full Frame`: disable extra ROI cropping for that class
- Preview / Export: - The right `Preview` panel runs preview inference after training - Toggle `Input` to start/stop live predictions; toggle `ROI` to switch between raw input and the thresholded mask view (shows dark/lum effects) - The slider bar under the preview image provides live `Dark Thresh` and `Lum Thresh` controls — adjust them to tune sign detection while watching the ROI view - `Export Model` writes model files and MCU helper files to the selected export folder
- Default output directories:
    - macOS: `~/Library/Application Support/TFLiteTraining/`
    - Windows: `%APPDATA%\\TFLiteTraining\\`
    - Structure:
        - `workspace/`: current project workspace
        - `datasets/`: training datasets (images grouped by class)
        - `runs/`: training runs and artifacts from each Train (e.g. .keras/.tflite/labels)
        - `logs/streamlit.log`: app logs (for debugging)
- Export to MCU:
    - `model.h / model.cpp`: drop into Arduino/ESP-IDF projects (array name can be set in Export)
    - `labels.txt`: class order (inference index mapping)

Quick Start Demo (10 minutes)

A step-by-step walkthrough that showcases every major feature. Ideal for presentations and first-time users.

Step 1 — Create a Project (30 sec)

1. Launch the app → you see the **Home** screen.
2. Click **Open Image Project** → **Start from empty**.
3. You are now in the workspace with three columns: **Classes**, **Training**, **Preview**.

Step 2 — Add Classes (30 sec)

1. In the left column, click **Add a class** to create a second class.
2. Click a class name to rename it (e.g. `SIGN`, `BACKGROUND`).
3. Each class card has three capture buttons: **Webcam**, **Device**, **Upload**.

> **Demo tip**: Create 2 classes — one for the sign, one for background/nothing.

Step 3 — Configure Device Source (1 min)

1. On a class card, click **Device** to open the device panel.
2. Click the ⚙ **gear icon** on the device panel to open settings:

Setting	Value	What it does
Image Size	96 / 160 / 384	Must match your firmware output
Color Mode	Grayscale / RGB	Grayscale for inference, RGB for data collection
Baud Rate	921600	Match your serial port speed
Sync Header	AA 55 AA	Frame start marker

3. Click **Apply** — the device preview should show live frames.

> **Showcase**: Change Image Size and see the preview adapt. This proves the app handles any resolution.

Step 4 — Capture Samples (2 min)

1. Click and **hold** the device/webcam capture button → samples are saved to that class.
2. Repeat for each class. Aim for 10-20 samples per class.
3. The sample count updates on each class card.

> **Showcase**: The device works with **any frame size** (96/160/384) and **Grayscale or RGB** — configured entirely from the gear icon, no code changes.

Step 5 — Tune Detection Thresholds (2 min) ⭐ KEY FEATURE

1. Click the **preprocess button** (🔍 icon) next to a class name.
2. The class edit page opens showing your samples on the left and a **Processed Preview** on the right.
3. The preview shows the **thresholded mask** — white = filtered out, dark = what the detector keeps.
4. Adjust the **Dark Thr** and **Lum Thr** sliders:

Slider	Default	Effect
Dark Thr	0	Pixels darker than this → ignored as shadow
Lum Thr	100	Pixels brighter than this → ignored as background

5. Watch the processed preview update in real-time as you drag the sliders.
6. The ROI bounding box (green outline) shows where the detector found the sign.

> **Showcase**: This is the core tuning workflow. The processed preview gives instant visual feedback — no guesswork.

Step 6 — Train the Model (1 min)

1. In the middle **Training** column, click **Train Model**.
2. To change model parameters, expand **Advanced**:

Parameter	Default	What it does
Image Size	96	All images resized to this before training
Batch Size	32	Samples per training step
Epochs	20	Training iterations
Learning Rate	0.0016	Optimization step size

3. Training takes 10-60 seconds. A progress bar shows status.

> **Showcase**: The **Image Size** parameter is independent of capture resolution. You can capture at 160×160 but train at 96×96 (or vice versa).

Step 7 — Preview & Tune Live (2 min) ⭐ KEY FEATURE

1. After training, the right **Preview** panel becomes active.
2. Toggle **Input ON** → live camera feed with predictions.
3. Toggle **ROI ON** → switches to the **thresholded mask view**.
4. Use the **slider bar** under the preview image:

```
Dark [===○========] 3    Lum [========○===] 76
```

5. Adjust sliders while watching the ROI view — the detection updates in real-time.

> **Showcase**: The slider bar + ROI toggle is the fastest way to find good thresholds. No need to re-train — just slide and see.

Step 8 — Save & Export (30 sec)

1. Click the **top-left menu** → **Save project** → saves a `.tmproj` file.
2. All settings are preserved: class names, samples, thresholds, training config.
3. Click **Export Model** → choose a folder → gets `.tflite`, `model.cpp`, `model.h`, `labels.txt`.

> **Showcase**: `.tmproj` is a complete snapshot. Share it with teammates or reopen it later — everything is restored.

Key Features Checklist

#	Feature	Where
1	Configurable device resolution (96/160/384)	Device gear ⚙
2	Grayscale / RGB color mode	Device gear ⚙
3	Per-class dark/lum threshold tuning	Class edit page (🔍)
4	Real-time processed preview with mask overlay	Class edit page
5	Live ROI toggle + slider bar	Preview panel
6	Image Size as training hyperparameter	Training → Advanced
7	Full project save/restore (.tmproj)	Top-left menu
8	MCU-ready export (.tflite + C sources)	Export button

ROI Detection Pipeline

The auto-crop uses a G-channel dark-object + edge detection algorithm to find signs:

1. **G-channel extraction**: green channel has best SNR in typical lighting
2. **Dark/Lum thresholding**: pixels with G between `Dark Thresh` (default 0) and `Lum Thresh` (default 100) are sign candidates; everything else → white
3. **Search window**: looks for the sign within x:10-90%, y:10-90% of the frame
4. **Edge detection**: within the search window, dark pixels near edges score higher
5. **Blob scoring**: best blob by weighted-sum × aspect-ratio × area × center-prior
6. **Crop**: from the original RGB using the detected bbox → BT.601 luminance → resize → contrast stretch

If no sign is found: center crop fallback (50%, 50% position, 40% side).

The **processed preview** (ROI toggle / class edit page) shows step 2: white = filtered out, dark = candidate pixels. Tune Dark/Lum thresholds in the slider bar to adjust what the detector considers a sign.

Export Behavior

- Export directory selection:
  - The app asks for an export folder from the workspace export entry points
  - The selected folder is reused as the last export location
- Overwrite behavior:
  - If export targets already exist, the app shows a confirmation dialog before overwriting
  - Cancel keeps the existing files unchanged
  - Confirm overwrites the existing export files in place
- Exported files:
  - Named model files:
    - `<export name>.tflite`
    - `<export name>_model_data.h`
    - `<export name>_model_data.cpp`
    - `model_settings.h`

        - `model_settings.cpp`
    - Compatibility files:
        - `model.h`
        - `model.cpp`
        - `labels.txt`
- Export naming:
    - `Export name` controls the `.tflite` base name and the generated `*_model_data.*` file names
    - `Array name` controls the C/C++ tensor array symbol used inside generated source files

Dataset Folder Structure

- The app reads classified datasets where each label maps to one class folder.
- Supported layouts:
    - `label/image`
    - `train/label/image` together with optional `val/label/image` and `test/label/image`
- Typical examples:

```
dataset/
├── cat/
│   ├── 0001.jpg
│   ├── 0002.png
│   └── ...
├── dog/
│   ├── 0001.jpg
│   ├── 0002.png
│   └── ...
├── labels.txt
└── labels.json
```

```
dataset/
├── train/
│   ├── cat/
│   │   ├── a.jpg
│   │   └── ...
│   └── dog/
│       ├── b.jpg
│       └── ...
├── val/
│   ├── cat/
│   └── dog/
└── test/
    ├── cat/
    └── dog/
```

- Notes:
    - `label` becomes the class name in the workspace.

- Supported image extensions are `jpg`, `jpeg`, `png`, `bmp`, `webp`, and `gif`.
- Imported webcam/upload samples may be stored as color images for display, but training is loaded as grayscale.
- Device samples are grayscale.
- Serial device frames use a sync header before the payload (96×96, 160×160, or 384×384, Grayscale or RGB, configurable in device settings). The default sync header is `AA 55 AA`, and it can be changed from the device settings gear if your firmware uses a different frame prefix.
- If the selected folder does not contain a supported classified dataset layout, the import page reports that no dataset was detected.

`.tmproj` File Structure

- `.tmproj` is a zip-based project archive used by `Save project` / `Open project`.
- It stores project state, samples, labels, and optional training artifacts.
- Typical structure:

```
project.tmproj
├── manifest.json
├── tm_classes.json
├── tm_train_latest.json
├── dataset/
│   ├── labels.txt
│   ├── labels.json
│   ├── Class 1/
│   │   ├── 8d3c....png
│   │   └── ...
│   ├── Class 2/
│   │   ├── 2a71....jpg
│   │   └── ...
│   └── ...
└── runs/
    └── latest/
        ├── model.keras
        ├── model.tflite
        ├── labels.txt
        └── ...
```

- Main files:
  - `manifest.json`: archive format/version and internal paths such as `dataset/` and `runs/latest/`
  - `tm_classes.json`: current class order plus per-class preprocess settings used by the workspace UI
  - `tm_train_latest.json`: latest trained model metadata and export references
  - `dataset/`: samples saved in dataset format, grouped by class
  - `processed_cache/`: cached processed previews regenerated from the current preprocess settings when available

    - `dataset/labels.txt` and `dataset/labels.json`: label list stored with the samples
    - `runs/latest/`: latest training outputs, included only when a trained run exists

- Compatibility notes:

    - `Open project` restores from the packaged `dataset/` directory.
    - If `tm_classes.json` is missing, the app can fall back to `dataset/labels.json` or `dataset/labels.txt`.
    - `.tmproj` should be treated as an app project archive, not as a generic dataset export.
    - `Save project` preserves the dataset-style sample layout so the archive can reopen with the same classes and samples.

FAQ

- macOS “App can’t be opened / unidentified developer”: right-click `TFLiteTraining.app` → Open → Confirm, or allow it in System Settings.
- macOS first-time webcam access: the app will trigger the system camera permission prompt. If you previously denied, re-enable it in “System Settings → Privacy & Security → Camera” for `TFLiteTraining.app`.
- macOS packaged app does not prompt: ensure you are running `dist/TFLiteTraining.app` (permissions are recorded per app bundle, not per terminal).
- Windows missing DLL: install Microsoft Visual C++ Redistributable 2015–2022 (x64).